

Now there is a different understanding about the system identification and digital twin part Can you go through these notes and explain again? Anomaly the process anomaly like tow pressure they meant in the document



Can you regenerate this diagram based on the scope change?



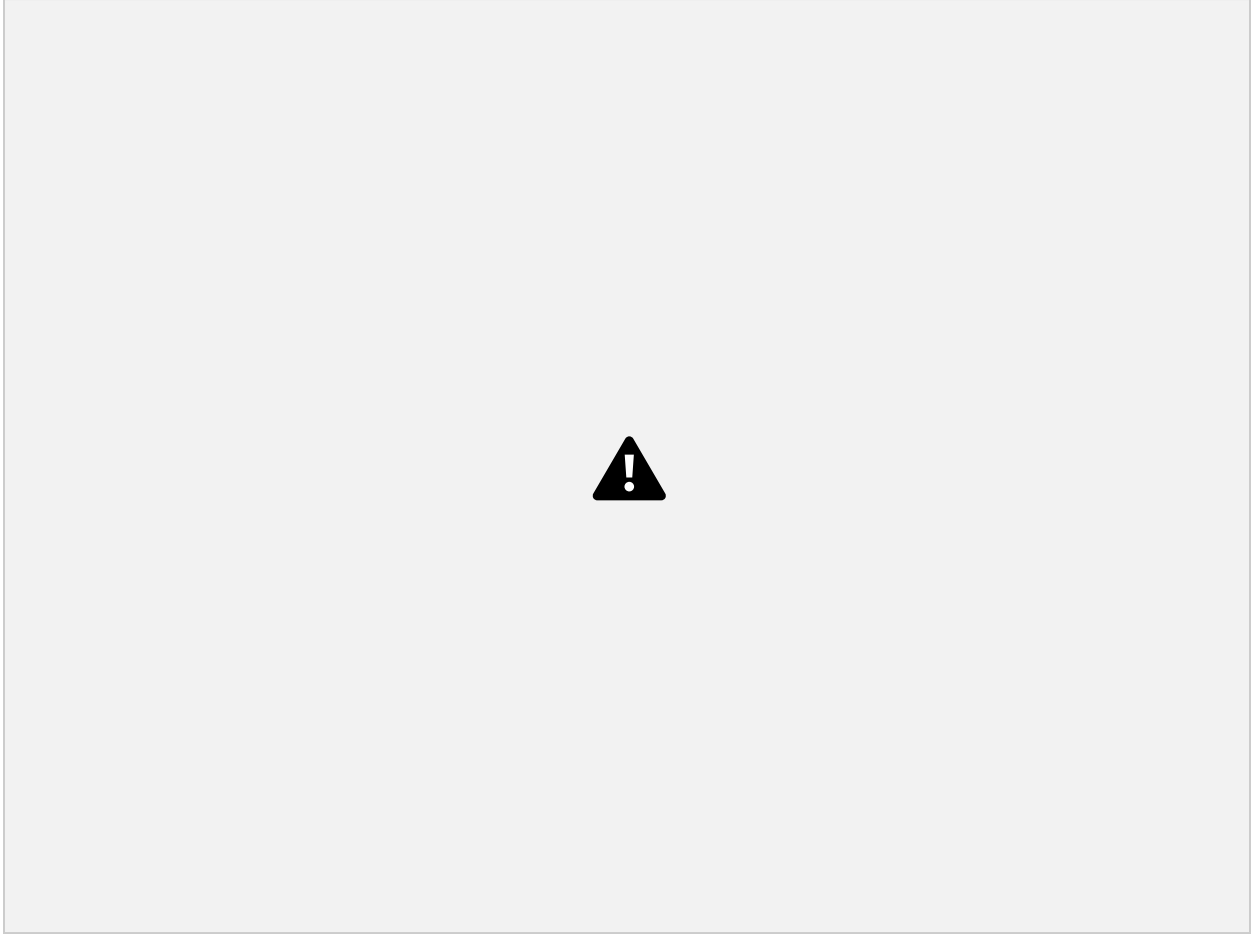
Can you regenerate this diagram based on the scope change?







Edit

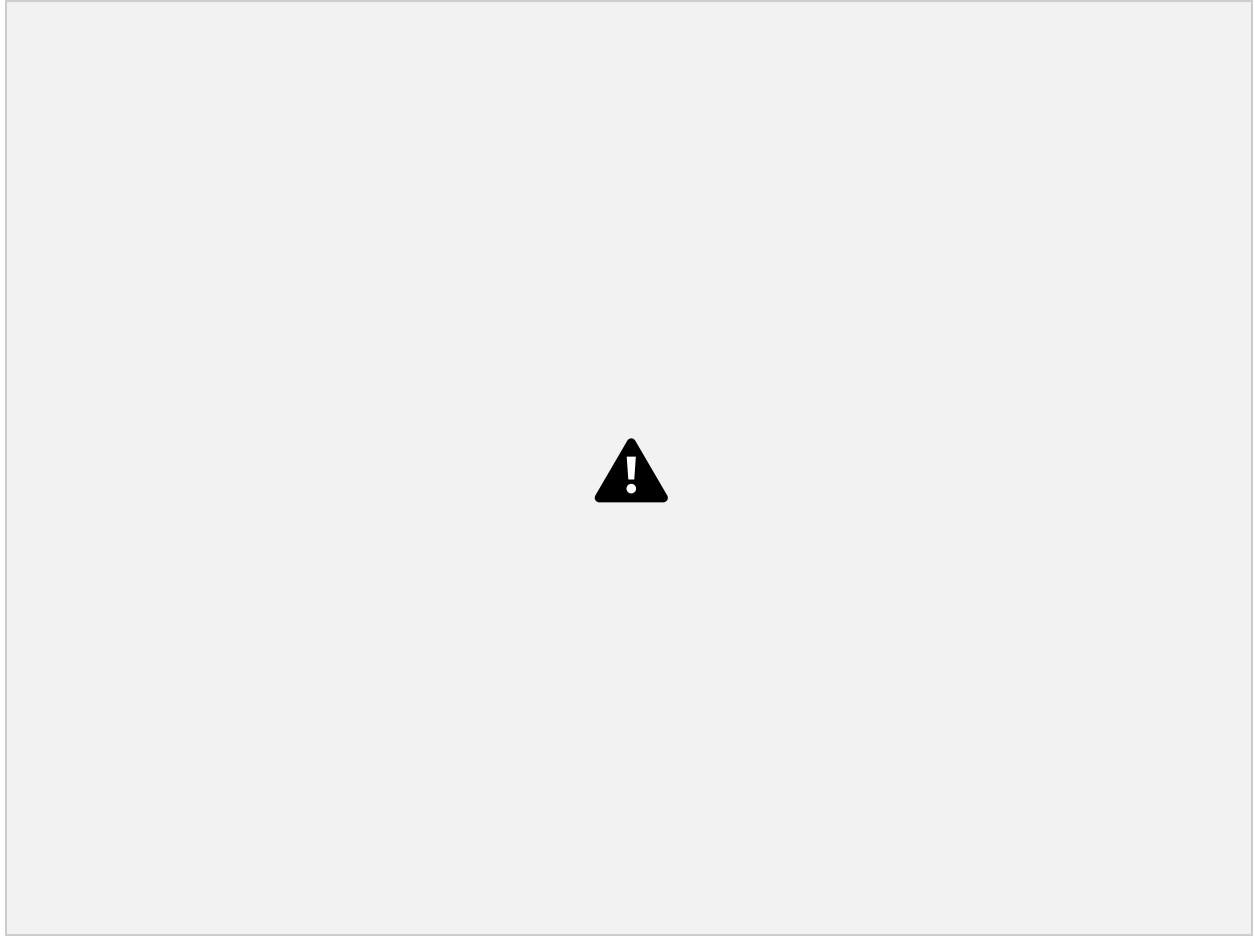


Can you update this too and regenerate the image based on the new scope





Edit



Can you please regenerate this too because of the change in scope? As per mom there is no vc and all





Edit

Revised WP5.3 Deliverables, Tasks & Approximate Timeline

(Aligned with Updated Scope — No Full VC / No Closed-Loop Autonomous Optimization)

D0 — Requirement Gathering & Industrial Alignment

Duration: ~1.5 Months

Objective

Define industrial scope, anomaly monitoring expectations, KPI requirements, and alignment with WP5.1/mTorres/OMTC.

D0.1 Stakeholder Requirement Collection

Duration: ~2 Weeks

Tasks

- Conduct meetings with OMTC, WP5.1, mTorres.
- Identify industrial pain points.
- Define expected monitoring capabilities.
- Identify anomaly use cases.
- Understand expected operator workflow.

Outputs

- Requirement specification
 - Use-case definition
-

D0.2 AFP/Laying Process Understanding

Duration: ~2 Weeks

Tasks

- Study AFP process stages:
 - approach
 - laydown
 - coursing
 - cut/add
 - retract
- Identify process variables and dependencies.
- Understand KPP/KPI definitions.

Outputs

- Process-state definition
 - Process variable mapping
-

D0.3 Scope Alignment & Validation

Duration: ~2 Weeks

Tasks

- Align scope boundaries between WP5.1 and WP5.3.
- Clarify:
 - behavioral process twin
 - anomaly monitoring
 - offline analysis framework
- Remove non-scope items:
 - full physics twin
 - heavy VC

- autonomous optimization

Outputs

- Approved scope baseline
-

D1 — System Architecture & Interface Definition

Duration: ~2 Months

D1.1 High-Level System Architecture

Duration: ~2 Weeks

Tasks

- Define overall architecture.
- Define:
 - ingestion flow
 - behavioral modeling flow
 - anomaly monitoring flow
 - visualization flow
- Define runtime vs offline workflows.

Outputs

- System architecture diagrams
-

D1.2 Functional Architecture Definition

Duration: ~3 Weeks

Tasks

Define major modules:

- ingestion
- behavioral modeling
- anomaly engine
- KPI framework
- analysis framework
- monitoring outputs

Outputs

- Functional architecture document
-

D1.3 Data Interface & Schema Definition

Duration: ~2 Weeks

Tasks

- Define interfaces with WP5.1.
- Define:
 - KPI schema
 - metadata structure
 - defect schema
 - process-state schema

Outputs

- ICD/interface document
-

D1.4 Monitoring & HMI Integration Definition

Duration: ~2 Weeks

Tasks

- Define dashboard outputs.
- Define anomaly visualization logic.
- Define operator alerts.

- Define expected process insight outputs.

Outputs

- HMI integration specification
-

D2 — Sample Data Collection & Modeling Preparation

Duration: ~2 Months

D2.1 Sample/Historical Data Collection

Duration: ~2 Weeks

Tasks

- Collect sample/historical mTorres datasets.
- Organize process datasets.
- Organize machine datasets.

Outputs

- Sample dataset repository
-

D2.2 KPI & SI Variable Definition

Duration: ~2 Weeks

Tasks

Define:

- process inputs
- process outputs

- prediction targets
- anomaly indicators
- KPI relationships

Outputs

- SI variable mapping
-

D2.3 Process-State & Label Definition

Duration: ~2 Weeks

Tasks

Define:

- process states
- operational phases
- defect labels
- event labels

Outputs

- Process labeling framework
-

D2.4 Modeling Dataset Structuring

Duration: ~2 Weeks

Tasks

- Create SI-ready datasets.
- Organize:
 - train
 - validation
 - test datasets
- Create time-window datasets.

Outputs

- Structured modeling datasets
-

D3 — Data Ingestion Pipeline Development

Duration: ~2 Months

D3.1 Ingestion Architecture Definition

Duration: ~2 Weeks

Tasks

- Define ingestion workflow.
- Define storage strategy.
- Define metadata handling.

Outputs

- Ingestion architecture
-

D3.2 Ingestion Pipeline Development

Duration: ~4 Weeks

Tasks

Develop:

- ingestion APIs
- parsers
- validation logic
- storage handling

Outputs

- Reusable ingestion framework
-

D3.3 Data Mapping for Behavioral Modeling

Duration: ~2 Weeks

Tasks

Map:

- KPIs
- defects
- process states
- contextual data
to modeling-ready structures.

Outputs

- Modeling-ready mapped datasets
-

D3.4 Mock Data Integration Testing

Duration: ~2 Weeks

Tasks

- Validate ingestion with sample data.
- Validate schema compatibility.
- Validate future WP5.1 compatibility.

Outputs

- Validated ingestion pipeline
-

D4 — Actual Laying Data Integration & Dataset Preparation

Duration: ~2.5 Months

D4.1 Actual Laying Data Collection

Duration: ~2 Weeks

Tasks

- Receive real laying data from WP5.1.
- Import through ingestion pipeline.
- Validate dataset completeness.

Outputs

- Integrated production datasets
-

D4.2 SI Variable & KPI Refinement

Duration: ~2 Weeks

Tasks

Refine:

- KPIs
- anomaly indicators
- prediction targets
- process-state definitions

using actual laying data.

Outputs

- Refined SI variable set

D4.3 Production Dataset Preparation

Duration: ~3 Weeks

Tasks

Prepare:

- runtime-ready datasets
- process-window datasets
- sequence datasets
- scenario datasets

Outputs

- Production-ready SI datasets
-

D4.4 Defect & Process Correlation Mapping

Duration: ~2 Weeks

Tasks

- Align defects with process conditions.
- Identify anomaly signatures.
- Map process instability regions.

Outputs

- Defect-process mapping
-

D4.5 Dataset Validation & Coverage Analysis

Duration: ~2 Weeks

Tasks

- Validate KPI coverage.
- Validate process-state coverage.
- Validate modeling readiness.

Outputs

- Dataset validation report
-

D5 — Behavioral Modeling & System Identification

Duration: ~3 Months

D5.1 Process Input/Output Mapping

Duration: ~2 Weeks

Tasks

Define:

- expected process behavior
- model inputs
- model outputs
- prediction targets

Outputs

- Behavioral mapping definition
-

D5.2 SI Framework Setup

Duration: ~2 Weeks

Tasks

Setup:

- MATLAB/Python SI environment
- training workflows
- evaluation workflows

Outputs

- SI development environment
-

D5.3 Behavioral Model Development

Duration: ~5 Weeks

Tasks

Develop:

- time-series models
- lightweight ML models
- behavioral process models

Outputs

- Behavioral process models
-

D5.4 Offline Validation (Validation Loop 1)

Duration: ~3 Weeks

Tasks

Perform:

- train/test validation
- residual analysis
- error computation
- model comparison

Outputs

- Offline validation report
-

D5.5 Runtime Drift & Anomaly Logic

Duration: ~2 Weeks

Tasks

Define:

- anomaly thresholds
- expected process envelopes
- runtime drift indicators

Outputs

- Runtime anomaly framework
-

D6 — Behavioral Process Twin & Offline Analysis Framework

Duration: ~3 Months

D6.1 Behavioral Process Twin Integration

Duration: ~3 Weeks

Tasks

Integrate:

- SI models
- prediction workflows

- expected behavior generation

Outputs

- Behavioral process twin
-

D6.2 Process Trend Prediction Framework

Duration: ~3 Weeks

Tasks

Predict:

- KPI trends
- expected signal behavior
- anomaly likelihood
- process envelopes

Outputs

- Prediction framework
-

D6.3 KPI Correlation Framework

Duration: ~2 Weeks

Tasks

Analyze:

- KPI interactions
- process dependencies
- parameter influence

Outputs

- KPI correlation framework

D6.4 Root Cause Analysis Framework

Duration: ~2 Weeks

Tasks

Develop offline engineering analysis framework for:

- defect relationships
- process instability analysis
- anomaly investigation

Outputs

- RCA framework
-

D6.5 Sensitivity Analysis Framework

Duration: ~2 Weeks

Tasks

Analyze:

- parameter sensitivity
- dominant process variables
- process influence

Outputs

- Sensitivity analysis framework
-

D6.6 What-If Simulation Capability

Duration: ~2 Weeks

Tasks

Simulate:

- process parameter variations
- expected KPI changes
- anomaly behavior

Outputs

- What-if analysis framework
-

D7 — Anomaly Detection & Monitoring Intelligence

Duration: ~2 Months

D7.1 Runtime Anomaly Detection Framework

Duration: ~3 Weeks

Tasks

Develop:

- deviation detection
- anomaly scoring
- process envelope monitoring

Outputs

- Runtime anomaly framework
-

D7.2 Process Stability Monitoring

Duration: ~2 Weeks

Tasks

Develop:

- process stability indicators
- health indicators
- drift monitoring

Outputs

- Stability monitoring framework
-

D7.3 Early Warning & Alert Logic

Duration: ~2 Weeks

Tasks

Develop:

- warning thresholds
- alert conditions
- notification logic

Outputs

- Alert framework
-

D7.4 Runtime Validation Loop (Validation Loop 2)

Duration: ~2 Weeks

Tasks

Compare:

- predicted vs actual behavior

- runtime error metrics
- model drift

Outputs

- Runtime validation framework
-

D8 — Visualization & WP5.1 Integration

Duration: ~2 Months

D8.1 Prediction Output Interface

Duration: ~2 Weeks

Tasks

Define:

- prediction outputs
- metadata schema
- KPI output structures

Outputs

- Prediction output interface
-

D8.2 WP5.1 Visualization Integration

Duration: ~3 Weeks

Tasks

Integrate:

- predicted KPIs

- anomaly indicators
- process insights
into WP5.1 visualization systems.

Outputs

- Visualization integration
-

D8.3 Dashboard & Monitoring Outputs

Duration: ~2 Weeks

Tasks

Provide:

- trend data
- anomaly indicators
- process envelopes
- operator insights

Outputs

- Dashboard-ready outputs
-

D8.4 Model Health & Confidence Indicators

Duration: ~2 Weeks

Tasks

Compute:

- model confidence
- prediction confidence
- runtime health metrics

Outputs

- Model-health framework
-

D9 — Continuous Learning & Model Improvement

Duration: ~1.5 Months

D9.1 Runtime Error Monitoring

Duration: ~2 Weeks

Tasks

Monitor:

- runtime prediction errors
- drift metrics
- anomaly false positives

Outputs

- Runtime error monitoring
-

D9.2 Incremental Model Improvement Workflow

Duration: ~2 Weeks

Tasks

Develop workflows for:

- model updates
- retraining
- recalibration

Outputs

- Model refinement workflow
-

D9.3 Continuous Learning Framework

Duration: ~2 Weeks

Tasks

Develop:

- runtime learning logic
- data update strategy
- improvement workflows

Outputs

- Continuous learning framework

In this this process anomaly detection, monitoring kpi changes, variations etc will be part of behaviour model (identified model?)

Yes — exactly.

Based on the updated scope, anomaly detection and KPI variation monitoring are now becoming a major responsibility of the **behavioral identified model**.

But there is an important architectural distinction.

Correct understanding now

The identified behavioral model itself is NOT directly:

"anomaly detector"

Instead:

It learns NORMAL process behavior.

Then anomaly detection is performed by:

Comparing:

Expected behavior (from identified model)

vs

Actual runtime behavior

So architecture becomes

Actual Process Data



System Identification

(learn normal behavior)



Behavioral Process Model



Predict Expected Process Behavior



Compare with Actual Runtime Data



Deviation / Drift / KPI Variation



Anomaly Detection

Example

Suppose identified model learned:

Speed = 200

Temperature = 72

Tow Pressure should remain 50 ± 3

Runtime

Actual machine:

Tow Pressure = 39

Then:

Deviation = High
→ Process anomaly

So YES:

These become part of the behavioral model workflow:

- KPI trend prediction
 - expected process envelope
 - process behavior prediction
 - runtime deviation monitoring
 - process drift detection
 - anomaly scoring
 - process stability monitoring
-

Important distinction

The SI/behavioral model gives:

Expected process behavior

The anomaly engine computes:

Difference between:
Expected vs Actual

So your architecture should now look like this

1. System Identification

Learns:

Normal process behavior

Outputs:

- behavioral model
 - expected KPI relations
 - expected signal trends
-

2. Behavioral Process Twin

Uses identified model to generate:

Expected process outputs

Expected KPI trends

Expected process envelopes

3. Runtime Monitoring & Anomaly Detection

Compares:

Actual runtime data

vs

Predicted/expected behavior

Computes:

- deviation
 - drift
 - anomaly score
 - KPI instability
-

So anomaly detection is now tightly coupled with:

behavioral model

AND

runtime comparison logic

Very important scope clarification

This is NOT classical anomaly detection like:

unsupervised AI anomaly detection only

Instead it is:

Model-based anomaly detection

Meaning:

Expected behavior is generated by the identified behavioral model.

Then:

Deviation from expected behavior

=

anomaly

So your D5/D6/D7 relationship now becomes

Deliverable

Role

D5

Learn behavioral process model

D6

Generate expected process behavior/trends

D7 Compare runtime behavior vs expected behavior

Runtime monitoring now mainly checks

KPI deviations

Example:

- pressure instability
 - abnormal tension
 - unusual vibration
 - temperature drift
-

Process-state abnormalities

Example:

- laydown phase taking too long
 - unexpected process transition
 - unstable process response
-

Signal behavior anomalies

Example:

- oscillations
 - spikes
 - drift
 - unexpected trend changes
-

This aligns VERY WELL with the MOM

Especially these points:

“Identification of anomalies in live data stream”

and

“variations in pressure or tension”

This is exactly:

behavioral-model-based monitoring

NOT:

- full AI autonomy
 - deep optimization control
 - advanced reinforcement learning
-

Final simplified understanding

System Identification

Learn how process normally behaves

Behavioral Process Twin

Predict how process SHOULD behave

Runtime Monitoring

Check whether actual process deviates from expected behavior

Anomaly Detection

Deviation beyond acceptable threshold

That is now the core of your updated WP5.3 scope.

runtime learning logic Is this required?. Because you mentioned previously that they need only offline

Yes — good catch.

Based on the updated MOM/scope, a full **runtime learning logic** is probably too strong and too research-oriented.

You should reduce it significantly.

What they MOST LIKELY want now

NOT this:

Online self-learning AI system
that continuously retrains itself automatically during production

That would be:

- risky industrially
 - difficult to validate
 - difficult to certify
 - not aligned with MOM
-

What they likely want instead

Runtime validation & monitoring

Meaning:

Monitor:

Predicted behavior

vs

Actual behavior

and then:

Offline engineering review

→ model improvement later

So the correct flow now becomes

Runtime (online)

Actual process data

- compare with expected behavior
- detect anomalies
- compute deviations
- generate alerts
- visualize KPIs

ONLY.

Then Offline

Later:

Collect runtime deviations

- engineer analyzes them
- retrain/refine model offline
- redeploy updated model

This is MUCH more realistic industrially.

So remove this wording

 Remove:

- runtime learning
- online learning
- self-learning system
- autonomous model adaptation
- continuous runtime retraining

Replace with

✓ Better wording:

- Runtime validation framework
- Model performance monitoring
- Drift monitoring
- Offline model refinement workflow
- Model recalibration workflow
- Runtime deviation monitoring

So D9 should probably become

D9 — Runtime Validation & Model Improvement Framework

NOT:

Continuous Learning Framework

Revised D9

D9 — Runtime Validation & Model Improvement Framework

Duration: ~1 Month

D9.1 Runtime Prediction Validation

Duration: ~2 Weeks

Tasks

- Compare predicted vs actual behavior.
- Compute runtime error metrics.
- Monitor KPI deviations.
- Detect model drift.

Outputs

- Runtime validation framework
-

D9.2 Offline Model Refinement Workflow

Duration: ~2 Weeks

Tasks

Define workflow for:

- collecting runtime deviations
- offline engineering review
- retraining/recalibration process
- updated model deployment

Outputs

- Model improvement workflow
-

VERY IMPORTANT difference

Before (too advanced)

Runtime:

Data → retrain model automatically

Now (industrial realistic)

Runtime:
Monitor only

Offline:
Improve model later

This aligns MUCH better with these MOM statements

“root cause analysis is post-process activity”

and

“details and analysis can be performed as a second step”

and

“industrial deployment focus”

Final recommendation

You should now avoid these terms in presentations:

 Avoid:

- autonomous learning
 - self-learning AI
 - online retraining
 - adaptive optimization
 - closed-loop autonomous AI
-

✓ Use:

- runtime validation
- deviation monitoring
- anomaly monitoring
- offline model refinement
- engineering feedback workflow
- model recalibration

This will align MUCH better with the actual industrial expectation.